Kevin Gómez Valderas 2ºDAM 24-25

ZOO Parte 3

Añadir funcionalidad

1. Añadir un modelo tipocomidaanimales, con un campo nombre. Los animales comen pienso, carne, gusanos, zanahorias, etc.

Modelo

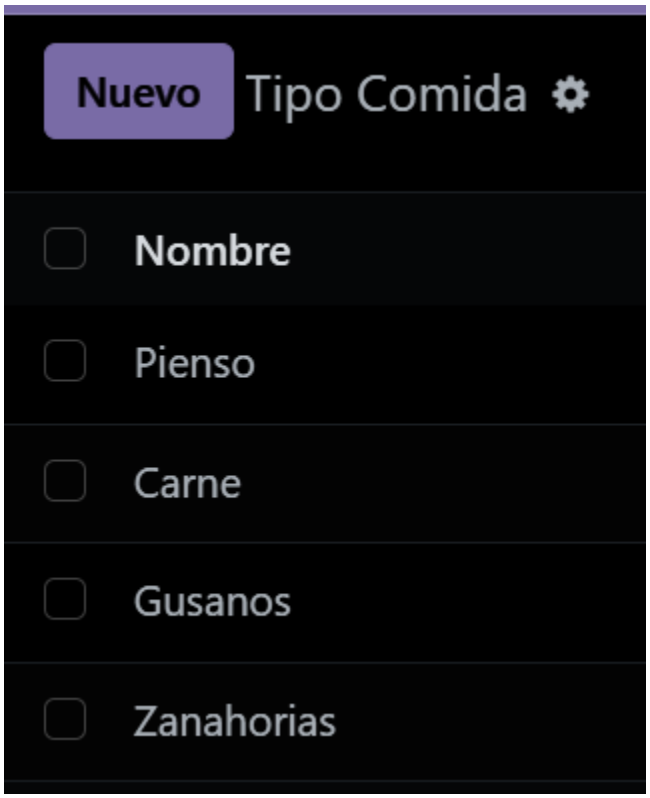
```python
class TipoComida(models.Model):
    _name = 'zoo.tipo.comida'
    _description = 'Tipos de comida para animales'

    name = fields.Char(string='Nombre', required=True)
```

Vista

```xml
<odoo>
    <record id="action_tipo_comida" model="ir.actions.act_window">
        <field name="name">Tipo Comida</field>
        <field name="res_model">zoo.tipo.comida</field>
        <field name="view_mode">tree,form</field>
        <field name="help" type="html">
            <p class="o_view_nocontent_smiling_face">
                Gestiona los tipos de comida para los animales.
            </p>
        </field>
    </record>

    <menuitem id="menu_tipo_comida"
        name="Tipo Comida"
        parent="menu_zoo_config"
        action="action_tipo_comida"
        sequence="90" />

    <record id="view_tipo_comida_search" model="ir.ui.view">
        <field name="name">zoo.tipo.comida.search</field>
        <field name="model">zoo.tipo.comida</field>
        <field name="arch" type="xml">
            <search>
                <field name="name" />
            </search>
        </field>
    </record>
```

```xml
    <!-- Vista de Árbol -->
    <record id="view_tipo_comida_tree" model="ir.ui.view">
        <field name="name">zoo.tipo.comida.tree</field>
        <field name="model">zoo.tipo.comida</field>
        <field name="arch" type="xml">
            <tree>
                <field name="name" />
            </tree>
        </field>
    </record>

    <!-- Vista de Formulario -->
    <record id="view_tipo_comida_form" model="ir.ui.view">
        <field name="name">zoo.tipo.comida.form</field>
        <field name="model">zoo.tipo.comida</field>
        <field name="arch" type="xml">
            <form>
                <field name="name" />
            </form>
        </field>
    </record>
</odoo>
```

Resultado



2. Añadir dieta recomendada especie. Se mostrará como lineas dentro de cada especie. Sus campos son el tipo de comida (referencia al modelo comida animales) y la cantidad en kilos. Con esto indicamos que le vamos a dar de comer a un animal de esa especie por defecto en un día. Por ejemplo a los camellos les podemos dar 5 kilos de zanahorias y 4 de puerros al día.

Modelo

Especies

```
dieta_recomendada_ids = fields.One2many('zoo.dieta.recomendada', 'especie_id', string="Dieta
recomendada")
```

DietaRecomendada

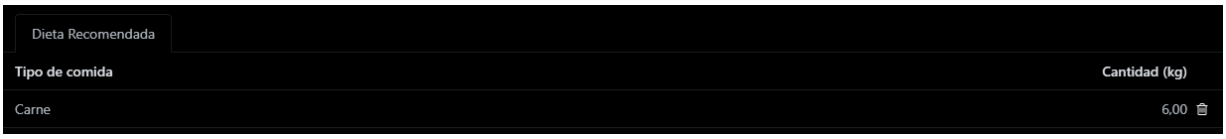
```
class DietaRecomendada(models.Model):
    _name = 'zoo.dieta.recomendada'
    _description = 'Dieta recomendada para cada especie'

    especie_id = fields.Many2one('zoo.especies', string="Especie", required=True,
ondelete='cascade')
    tipo_comida_id = fields.Many2one('zoo.tipo.comida', string="Tipo de comida", required=True)
    cantidad_kg = fields.Float(string="Cantidad (kg)", required=True, help="Cantidad diaria
recomendada en kilogramos")
```

Vista

Especies

```
<notebook>
    <page string="Dieta Recomendada">
        <field name="dieta_recomendada_ids">
            <tree editable="bottom">
                <field name="tipo_comida_id" />
                <field name="cantidad_kg" />
            </tree>
        </field>
    </page>
</notebook>
```

Resultado



3. Un botón en los continentes que de un mensaje indicando de cuantas especies diferentes de este continente hay en el zoo (puedes mostrar un mensaje como con el siguiente ejempo

from odoo.exceptions import ValidationError, UserError
def boton_aviso(self):
raise UserError('Cuidaaaoooo')

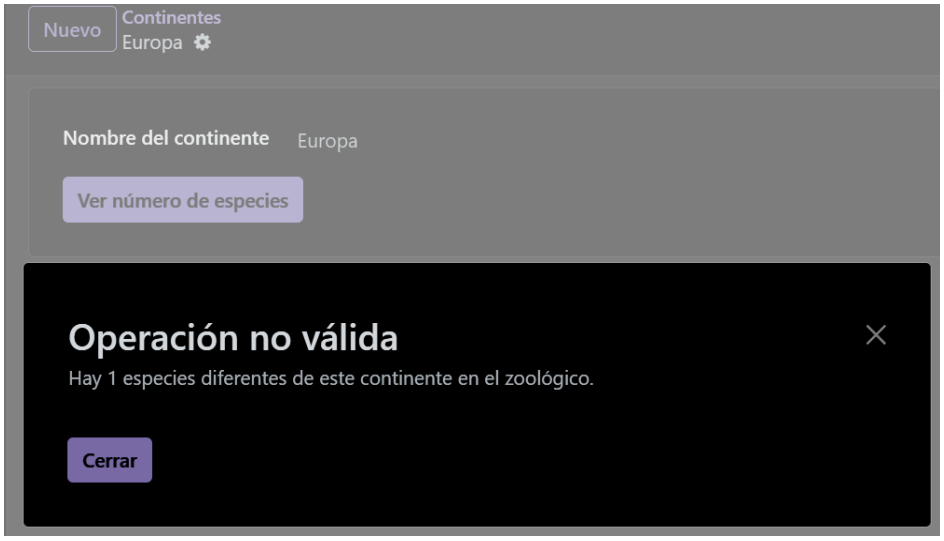
Model

```
especie_count = fields.Integer(string='Número de especies', compute='_compute_especie_count')

def _compute_especie_count(self):
    for record in self:
        record.especie_count = self.env['zoo.especies'].search_count([('continente_ids', '=',
record.id)])

def boton_aviso(self):
    for record in self:
        message = f'Hay {record.especie_count} especies diferentes de este continente en el
zoológico.'
        raise UserError(message)
```

Vista

```
<div style="width: 200px;" >
    <button name="boton_aviso"
        type="object"
        string="Ver número de especies" />
</div>
```

Resultado



4.

1. Añadir campo salario base al modelo con los estados laborales.

2. Mostrar un campo salario en el modelo de empleados donde el salario depende del estado laboral y de los estudios. El salario aumenta un 10% si se tiene la ESO y un 50 % si se tienen estudios superiores.

Modelos

EstadosLaborales

```
salario_base = fields.Float(string='Salario Base', required=True)
```

Cuidadores

```
# Nuevo campo computado para el salario
salario = fields.Float(
    string='Salario',
    compute='_compute_salario',
    store=True
)

@api.depends('estado_laboral_id', 'estudios')
def _compute_salario(self):
    for record in self:
        salario_base = record.estado_laboral_id.salario_base if record.estado_laboral_id else
0.0
        incremento = 1.0  # Sin estudios, sin incremento

        if record.estudios == 'eso':
            incremento += 0.10  # Aumento del 10% si tiene ESO
        elif record.estudios in ['ciclo_grado_superior', 'universitario']:
            incremento += 0.50  # Aumento del 50% si tiene estudios superiores

        record.salario = salario_base * incremento
```

Vista

EstadosLaborales

```
<field name="salario_base" string="Salario Base"/>
```

Cuidadores

```
<field name="salario" string="Salario" />
```

Resultado

Nuevo
Estados Laborales
Prácticas
Estado Laboral
Prácticas
Orden
1
Salario Base
500,00

Nuevo
Cuidadores
Kevin
1 / 1
Siguiente Estado
Prácticas
Tiempo Parcial
Falso Autónomo
Nombre Kevin
Kevin
Fecha de Nacimiento
17/02/2002
Teléfono Kevin
601479874
Salario
500,00
Dirección
Calle Espliego 6
Experiencia Laboral
Camarero
Estudios Kevin
ESO
Bachillerato
Ciclo Grado Medio
Ciclo Grado Superior
Universitario

3. Añadir un botón en el formulario de estados laborales que calcule la masa salarial total para todos los empleados (lo que cobran todos los empleados juntos teniendo el cuenta el incremento por estudios).

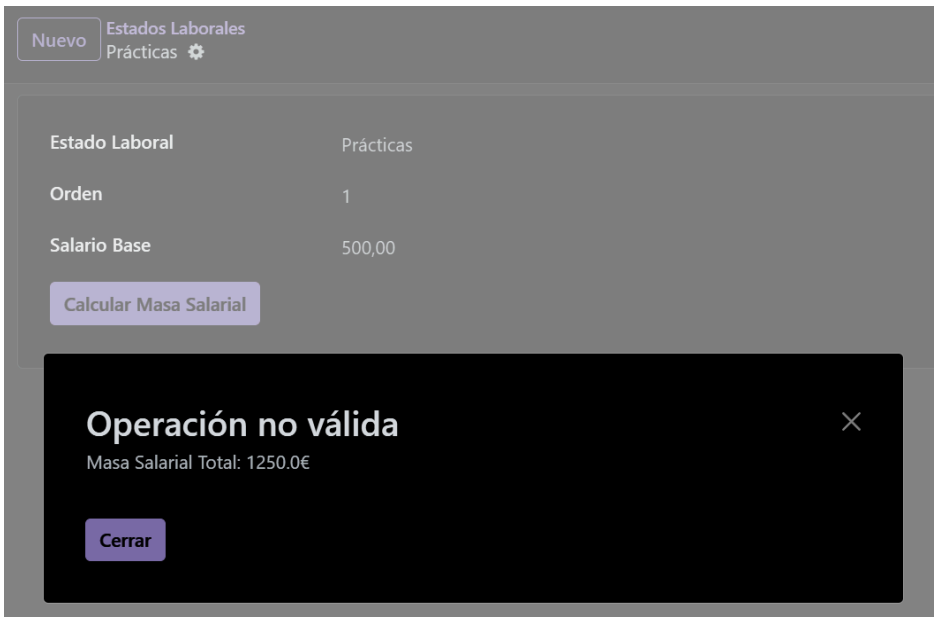
Modelo

```python
masa_salarial = fields.Float(string="Masa Salarial Total", compute="calcular_masa_salarial",
store=False)

def calcular_masa_salarial(self):
    """
    Método para calcular la masa salarial total de todos los cuidadores
    teniendo en cuenta el incremento por estudios.
    """
    # Obtener todos los cuidadores que tengan este estado laboral
    cuidadores = self.env['zoo.cuidadores'].search([('estado_laboral_id', '=', self.id)])

    # Calcular la masa salarial
    masa_salarial = sum(cuidador.salario for cuidador in cuidadores)

    # Lanzar un error de usuario con el resultado
    raise UserError("Masa Salarial Total: " + str(masa_salarial) + "€")
```

Vista

```html
<div style="width: 200px;">
    <button name="calcular_masa_salarial" type="object" string="Calcular Masa Salarial"
class="oe_highlight"/>
</div>
```

Resultado



4. Añadir campo en el modelo de estados laborales donde se muestre la media de los salarios de los empleados para ese estado laboral.

Modelo

```python
# Campo de la media de salarios
media_salarios = fields.Float(string="Media de Salarios", compute="_compute_media_salarios",
store=False)

def _compute_media_salarios(self):
    """
    Método para calcular la media de los salarios de los empleados
    para el estado laboral actual.
    """
    # Obtener todos los cuidadores que tengan este estado laboral
    cuidadores = self.env['zoo.cuidadores'].search([('estado_laboral_id', '=', self.id)])

    # Si no hay cuidadores, la media será 0
    if not cuidadores:
        self.media_salarios = 0.0
    else:
        # Sumar los salarios de todos los cuidadores
        total_salarios = sum(cuidador.salario for cuidador in cuidadores)
        # Calcular la media
        self.media_salarios = total_salarios / len(cuidadores)
```

Vista

```xml
<field name="media_salarios" />
```

Resultado

5. Añadir botón cría en animales. Crea un nuevo animal de nombre “Cría de” y el nombre del animal activo. Todos los datos son iguales excepto el estado (normal) y la fecha de llegada y nacimiento (fecha de hoy)

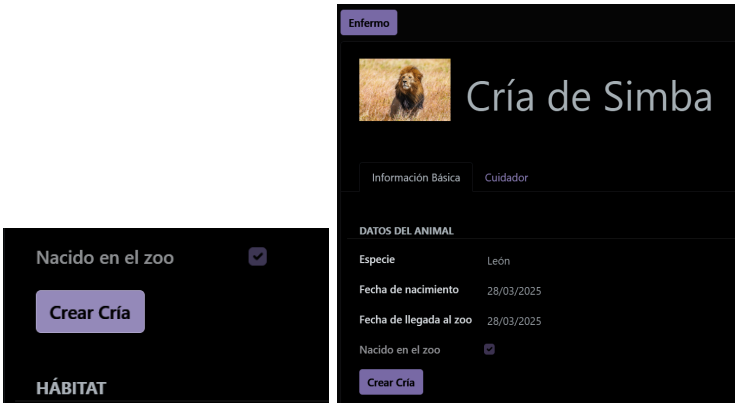
Modelo

```python
def action_cria(self):
    for animal in self:
        # Crear el nuevo animal (cría)
        cria = self.env['zoo.animales'].create({
            'name': f'Cría de {animal.name}',
            'especie_id': animal.especie_id.id,
            'habitaculo_id': animal.habitaculo_id.id,
            'fecha_llegada': fields.Date.today(),
            'fecha_nacimiento': fields.Date.today(),
            'nacido_zoo': True,
            'cuidador_id': animal.cuidador_id.id,
            'state': 'normal',
        })
        # Retornar la acción para abrir la vista del formulario del nuevo animal
        return {
            'type': 'ir.actions.act_window',
            'res_model': 'zoo.animales',
            'res_id': cria.id,
            'view_mode': 'form',
            'view_type': 'form',
            'target': 'current',  # Abrir en la ventana actual
        }
```

Vista

```html
<div style="width: 100px">
    <button name="action_cria" string="Crear Cría" type="object" class="oe_highlight"/>
</div>
```

Resultado



6.

1. Añadir un campo con el máximo de animales para el modelo habitáculo.

2. Al cambiar de habitáculo de un animal si sobrepasamos el total de animales en este habitáculo que nos de un warning. Hacerlo con read_count (tener cuidado porque el animal en que acabamos de cambiar no está en la base de datos como en este habitáculo y no lo contará en el read_count).

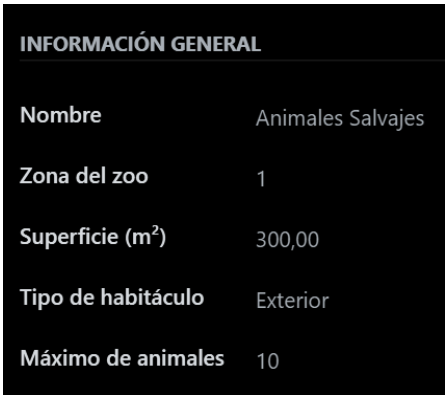
Modelo

```python
@api.model
def create(self, vals):
    # Crear el animal
    animal = super(Animales, self).create(vals)
    # Verificar si el habitáculo alcanza el límite
    if animal.habitaculo_id.total_animales > animal.habitaculo_id.max_animales:
        raise ValidationError('El habitáculo ha alcanzado el máximo de animales permitidos.')
    return animal

def write(self, vals):
    if 'habitaculo_id' in vals:
        # Obtener el habitáculo actual y el nuevo
        habitaculo_anterior = self.habitaculo_id
        habitaculo_nuevo = self.env['zoo.habitaculos'].browse(vals['habitaculo_id'])

        # Comprobar si el cambio supera el máximo de animales
        if habitaculo_nuevo.total_animales + 1 > habitaculo_nuevo.max_animales:
            raise ValidationError('El habitáculo ha alcanzado el máximo de animales
permitidos.')

    # Realizar la escritura
    return super(Animales, self).write(vals)
```

Vista

```xml
<field name="max_animales" />
```

Resultado



Nuevo Animales

Buscar...

1-10 / 10

Nombre del animal	Especie	Nacido en el zoo	Estado	Habitáculo	Cuidador responsable
Simba	León	✓	En recuperación	Animales Salvajes	Kevin
Cría de Simba	León	✓	Normal	Animales Salvajes	Kevin
Cría de Simba	León	✓	Normal	Animales Salvajes	Kevin
Cría de Cría de Simba	León	✓	Normal	Animales Salvajes	Kevin
Cría de Cría de Cría de Simba	León	✓	Normal	Animales Salvajes	Kevin
Cría de Cría de Cría de Cría de Simba	León	✓	Normal	Animales Salvajes	Kevin
Cría de Cría de Cría de Cría de Cría de Simba	León	✓	Normal	Animales Salvajes	Kevin
Cría de Cría de Cría de Cría de Cría de Cría de Simba	León	✓	Normal	Animales Salvajes	Kevin
Cría de Cría de Cría de Cría de Cría de Cría de Cría de Simba	León	✓	Normal	Animales Salvajes	Kevin
Cría de Cría de Cría de Cría de Cría de Cría de Cría de Cría de Simba	León	✓	Normal	Animales Salvajes	Kevin

Error de validación

El habitáculo ha alcanzado el máximo de animales permitidos.

Cerrar

7. Wizard “Cambiar especie de habitáculo”. Indicamos una especie y un habitáculo de origen y uno de destino. Cambiamos el habitáculo a todos los animales de esa especie que estaban en el habitáculo de origen por el de destino.

Modelo

ZooCambioHabitaculoWizard

```
class CambioHabitaculoWizard(models.TransientModel):
    _name = 'zoo.cambio.habitaculo.wizard'
    _description = 'Wizard para cambiar el habitáculo de una especie'

    especie_id = fields.Many2one('zoo.especies', string="Especie", required=True)
    habitaculo_origen_id = fields.Many2one('zoo.habitaculos', string="Habitáculo de Origen",
required=True)
    habitaculo_destino_id = fields.Many2one('zoo.habitaculos', string="Habitáculo de Destino",
required=True)

    def action_cambiar_habitaculo(self):
        """ Cambia los animales de una especie del habitáculo de origen al de destino """
        animales = self.env['zoo.animales'].search([
            ('especie_id', '=', self.especie_id.id),
            ('habitaculo_id', '=', self.habitaculo_origen_id.id)
        ])
        for animal in animales:
            animal.habitaculo_id = self.habitaculo_destino_id
        return {'type': 'ir.actions.act_window_close'}

    def action_cancel(self):
        """ Cierra el wizard sin hacer cambios """
        return {'type': 'ir.actions.act_window_close'}
```

Habitaculos

```
def action_abrir_wizard_cambio_habitaculo(self):
    """ Abre el wizard para cambiar el habitáculo de una especie """
    return {
        'name': 'Cambiar Especie de Habitáculo',
        'type': 'ir.actions.act_window',
        'res_model': 'zoo.cambio.habitaculo.wizard',
        'view_mode': 'form',
        'target': 'new',
    }
```

Vista

CambioHabitaculo

```
<odoo>
    <record id="action_zoo_cambio_habitaculo_wizard" model="ir.actions.act_window">
        <field name="name">Cambiar Especie de Habitáculo</field>
        <field name="res_model">zoo.cambio.habitaculo.wizard</field>
        <field name="view_mode">form</field>
        <field name="target">new</field>
    </record>

    <record id="view_zoo_cambio_habitaculo_wizard" model="ir.ui.view">
        <field name="name">zoo.cambio.habitaculo.wizard.form</field>
        <field name="model">zoo.cambio.habitaculo.wizard</field>
        <field name="arch" type="xml">
            <form string="Cambiar Habitáculo de Especie">
                <group>
                    <field name="especie_id" />
                    <field name="habitaculo_origen_id" />
                    <field name="habitaculo_destino_id" />
                </group>
                <footer>
                    <button string="Cambiar Habitáculo" type="object"
                        name="action_cambiar_habitaculo" class="btn-primary" />
                    <button string="Cancelar" type="object" name="action_cancel"
class="btn-default" />
                </footer>
            </form>
        </field>
    </record>
</odoo>
```

Habitaculos

```
<div style="width: 300px;">
    <button name="action_abrir_wizard_cambio_habitaculo"
        string="Cambiar Especie de Habitáculo"
        type="object"
        class="btn-primary" />
</div>
```

CSV

```
access_zoo_cambio_habitaculo_wizard,zoo.cambio.habitaculo.wizard,model_zoo_cambio_habitaculo_wi
zard,base.group_user,1,1,1,1
```

Resultado

Nombre del animal	Especie	Nacido en el zoo	Estado	Habitáculo	Cuidador responsable
Simba	León	☑	En recuperación	Animales Salvajes	Kevin
Cría de Simba	León	☑	Normal	Animales Salvajes	Kevin
Cría de Simba	León	☑	Normal	Animales Salvajes	Kevin
Cría de Cría de Simba	León	☑	Normal	Animales Salvajes	Kevin
Cría de Cría de Cría de Simba	León	☑	Normal	Animales Salvajes	Kevin
Cría de Cría de Cría de Cría de Simba	León	☑	Normal	Animales Salvajes	Kevin
Cría de Cría de Cría de Cría de Cría de Simba	León	☑	Normal	Animales Salvajes	Kevin
Cría de Cría de Cría de Cría de Cría de Cría de Simba	León	☑	Normal	Animales Salvajes	Kevin
Cría de Cría de Cría de Cría de Cría de Cría de Cría de Simba	León	☑	Normal	Animales Salvajes	Kevin
Cría de Cría de Cría de Cría de Cría de Cría de Cría de Cría de Simba	León	☑	Normal	Animales Salvajes	Kevin

Cambiar Especie de Habitáculo

Cambiar Especie de Habitáculo

Especie León

Habitáculo de Origen Animales Salvajes

Habitáculo de Destino Animales Pasivos

Cambiar Habitáculo **Cancelar**

Nombre del animal	Especie	Nacido en el zoo	Estado	Habitáculo	Cuidador responsable
Simba	León	☑	En recuperación	Animales Pasivos	Kevin
Cría de Simba	León	☑	Normal	Animales Pasivos	Kevin
Cría de Simba	León	☑	Normal	Animales Pasivos	Kevin
Cría de Cría de Simba	León	☑	Normal	Animales Pasivos	Kevin
Cría de Cría de Cría de Simba	León	☑	Normal	Animales Pasivos	Kevin
Cría de Cría de Cría de Cría de Simba	León	☑	Normal	Animales Pasivos	Kevin
Cría de Cría de Cría de Cría de Cría de Simba	León	☑	Normal	Animales Pasivos	Kevin
Cría de Cría de Cría de Cría de Cría de Cría de Simba	León	☑	Normal	Animales Pasivos	Kevin
Cría de Cría de Cría de Cría de Cría de Cría de Cría de Simba	León	☑	Normal	Animales Pasivos	Kevin
Cría de Cría de Cría de Cría de Cría de Cría de Cría de Cría de Simba	León	☑	Normal	Animales Pasivos	Kevin

8. Wizard eliminar alimento de dieta de especies. Indicamos uno o varios alimentos y una o varias especies los eliminamos de todas las dietas de estas especies.

Modelo

EliminarAlimento

```
class EliminarAlimentoWizard(models.TransientModel):
    _name = 'zoo.eliminar.alimento.wizard'
    _description = 'Wizard para eliminar alimentos de la dieta de especies'

    especie_ids = fields.Many2many('zoo.especies', string="Especies", required=True)
    tipo_comida_ids = fields.Many2many('zoo.tipo.comida', string="Tipos de comida",
required=True)

    def action_eliminar_alimentos(self):
        """ Elimina los alimentos seleccionados de las dietas de las especies elegidas """
        for especie in self.especie_ids:
            dietas_a_eliminar = self.env['zoo.dieta.recomendada'].search([
                ('especie_id', '=', especie.id),
                ('tipo_comida_id', 'in', self.tipo_comida_ids.ids)
            ])
            dietas_a_eliminar.unlink()  # Elimina las dietas filtradas

        return {'type': 'ir.actions.act_window_close'}

    def action_cancel(self):
        """ Cierra el wizard sin hacer cambios """
        return {'type': 'ir.actions.act_window_close'}
```

Especies

```
def action_abrir_wizard_eliminar_alimentos(self):
    """ Abre el wizard para eliminar alimentos de la dieta """
    return {
        'type': 'ir.actions.act_window',
        'res_model': 'zoo.eliminar.alimento.wizard',
        'view_mode': 'form',
        'target': 'new',
    }
```

Vista

EliminarAlimento

```
<odoo>
    <record id="action_zoo_eliminar_alimento_wizard" model="ir.actions.act_window">
        <field name="name">Eliminar Alimentos de Dieta</field>
        <field name="res_model">zoo.eliminar.alimento.wizard</field>
        <field name="view_mode">form</field>
        <field name="target">new</field>
```

```xml
    </record>

    <record id="view_zoo_eliminar_alimento_wizard" model="ir.ui.view">
        <field name="name">zoo.eliminar.alimento.wizard.form</field>
        <field name="model">zoo.eliminar.alimento.wizard</field>
        <field name="arch" type="xml">
            <form string="Eliminar Alimentos de la Dieta">
                <group>
                    <field name="especie_ids" widget="many2many_tags" />
                    <field name="tipo_comida_ids" widget="many2many_tags" />
                </group>
                <footer>
                    <button name="action_eliminar_alimentos" string="Eliminar" type="object"
                        class="btn-danger" />
                    <button name="action_cancel" string="Cancelar" class="btn-secondary"
                        special="cancel" />
                </footer>
            </form>
        </field>
    </record>
</odoo>
```

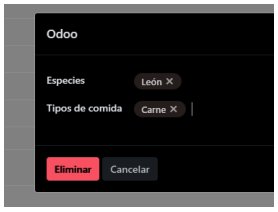
Especies

```xml
<div style="width: 300px;">
    <button name="action_abrir_wizard_eliminar_alimentos"
        string="Eliminar Alimentos de Dieta"
        type="object"
        class="btn-danger" />
</div>
```

CSV

```csv
access_zoo_eliminar_alimento_wizard,zoo.eliminar.alimento.wizard,model_zoo_eliminar_alimento_wizard,base.group_user,1,1,1,1
```

Resultado



9. Wizard añadir continente. Seleccionamos una serie de especies y un continente. Se añada el continente a todas las especies.

Modelo

WizardAnadirContinente

```
class WizardAnadirContinente(models.TransientModel):
    _name = 'zoo.anadir.continente.wizard'
    _description = 'Wizard para añadir continente a especies'


    especie_ids = fields.Many2many('zoo.especies', string="Especies", required=True)
    continente_ids = fields.Many2one('zoo.continentes', string='Continente a añadir',
required=True)

    def action_add_continent(self):
        """Asocia el continente seleccionado a las especies seleccionadas sin borrar los
continentes anteriores"""
        for especie in self.especie_ids:
            # Verificar si el continente ya está asociado
            if self.continente_ids not in especie.continente_ids:
                # Añadir el continente al campo Many2many
                especie.write({
                    'continente_ids': [(4, self.continente_ids.id)]
                })
        return {'type': 'ir.actions.act_window_close'}

    def action_cancel(self):
        """Cierra el wizard sin hacer cambios"""
        return {'type': 'ir.actions.act_window_close'}
```

Si queremos sobreescribir los continentes

```
def action_add_continent(self):
    """Asocia el continente seleccionado a las especies seleccionadas"""
    for especie in self.especie_ids:
        especie.continente_ids = self.continente_ids
    return {'type': 'ir.actions.act_window_close'}
```

Especies

```
def action_abrir_wizard_añadir_continente(self):
    """ Abre el wizard para añadir continente a las especies seleccionadas """
    return {
        'type': 'ir.actions.act_window',
        'res_model': 'zoo.anadir.continente.wizard',
        'view_mode': 'form',
        'target': 'new',
    }
```

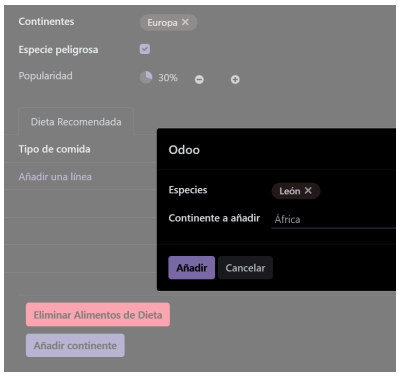
Vista

```xml
<odoo>
    <record id="view_zoo_anadir_continente_wizard_form" model="ir.ui.view">
        <field name="name">zoo.anadir.continente.wizard.form</field>
        <field name="model">zoo.anadir.continente.wizard</field>
        <field name="arch" type="xml">
            <form string="Añadir continente a especies">
                <sheet>
                    <group>
                        <field name="especie_ids" widget="many2many_tags" />
                        <field name="continente_ids" options="{'no_create': True}" />
                    </group>
                    <footer>
                        <button string="Añadir" type="object" name="action_add_continent"
                            class="btn-primary" />
                        <button string="Cancelar" type="object" name="action_cancel"
                            class="btn-secondary" />
                    </footer>
                </sheet>
            </form>
        </field>
    </record>

    <!-- Acción de menú para abrir el wizard -->
    <record id="action_abrir_wizard_añadir_continente" model="ir.actions.act_window">
        <field name="name">Añadir continente a especies</field>
        <field name="res_model">zoo.anadir.continente.wizard</field>
        <field name="view_mode">form</field>
        <field name="target">new</field>
    </record>
</odoo>
```

CSV

```csv
access_zoo_anadir_continente_wizard,zoo.anadir.continente.wizard,model_zoo_anadir_continente_wizard,base.group_user,1,1,1,1
```

Resultado



10. Wizard añadir alimento a la dieta. Indicamos una o varias especies, un alimento y una cantidad y añadimos este alimento con la cantidad a la dieta de las especies indicadas.

Modelo

AgregarAlimentoWizard

```python
class AgregarAlimentoWizard(models.TransientModel):
    _name = 'zoo.agregar.alimento.wizard'
    _description = 'Wizard para agregar alimentos a la dieta de especies'

    especie_ids = fields.Many2many('zoo.especies', string="Especies", required=True)
    tipo_comida_ids = fields.Many2many('zoo.tipo.comida', string="Tipos de comida", required=True)
    cantidad = fields.Float(string="Cantidad", required=True)

    def action_agregar_alimentos(self):
        """ Agrega los alimentos seleccionados a las dietas de las especies elegidas """
        dieta_model = self.env['zoo.dieta.recomendada']

        for especie in self.especie_ids:
            for tipo_comida in self.tipo_comida_ids:
                # Buscar si ya existe un registro de dieta para esta especie y alimento
                dieta_existente = dieta_model.search([
                    ('especie_id', '=', especie.id),
                    ('tipo_comida_id', '=', tipo_comida.id)
                ], limit=1)

                if dieta_existente:
                    # Si ya existe, actualizar la cantidad
                    dieta_existente.sudo().write({'cantidad': dieta_existente.cantidad_kg + self.cantidad})
                else:
                    # Si no existe, crear una nueva dieta
                    dieta_model.sudo().create({
                        'especie_id': especie.id,
                        'tipo_comida_id': tipo_comida.id,
                        'cantidad_kg': self.cantidad
                    })

        return {'type': 'ir.actions.act_window_close'}

    def action_cancel(self):
        """ Cierra el wizard sin hacer cambios """
        return {'type': 'ir.actions.act_window_close'}
```

Especies

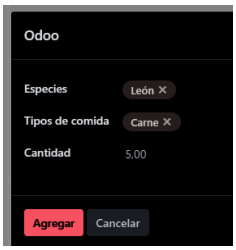
```python
def action_abrir_wizard_agregar_alimento(self):
    """ Abre el wizard para agregar alimento a las especies seleccionadas """
    return {
        'type': 'ir.actions.act_window',
        'res_model': 'zoo.agregar.alimento.wizard',
        'view_mode': 'form',
        'target': 'new',
    }
```

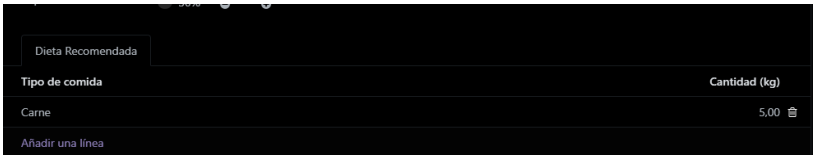
Vista

```xml
<odoo>
    <record id="action_zoo_agregar_alimento_wizard" model="ir.actions.act_window">
        <field name="name">Eliminar Alimentos de Dieta</field>
        <field name="res_model">zoo.agregar.alimento.wizard</field>
        <field name="view_mode">form</field>
        <field name="target">new</field>
    </record>

    <record id="view_zoo_agregar_alimento_wizard" model="ir.ui.view">
        <field name="name">zoo.agregar.alimento.wizard.form</field>
        <field name="model">zoo.agregar.alimento.wizard</field>
        <field name="arch" type="xml">
            <form string="Agregar Alimentos de la Dieta">
                <group>
                    <field name="especie_ids" widget="many2many_tags" />
                    <field name="tipo_comida_ids" widget="many2many_tags" />
                    <field name="cantidad" />
                </group>
                <footer>
                    <button name="action_agregar_alimentos" string="Agregar" type="object"
                        class="btn-danger" />
                    <button name="action_cancel" string="Cancelar" class="btn-secondary"
                        special="cancel" />
                </footer>
            </form>
        </field>
    </record>
</odoo>
```

```csv
access_zoo_agregar_alimento_wizard,zoo.agregar.alimento.wizard,model_zoo_agregar_alimento_wizard,base.group_user,1,1,1,1
```

Resultado





11. Wizard para dar de alta grupos de animales. Indicamos todos los datos de un animal y un número de animales que forman el grupo. Se añaden todos llamándolos Nombre 1, Nombre 2, etc. La fecha de llegada es la fecha de hoy. Se deja vacía la fecha de nacimiento.

Modelo
AltaAnimalesWizard

```python
class AltaAnimalesWizard(models.TransientModel):
    _name = 'zoo.alta.animales.wizard'
    _description = 'Wizard para alta de grupos de animales'

    name = fields.Char(string='Nombre base', required=True)
    especie_id = fields.Many2one('zoo.especies', string='Especie', required=True)
    habitaculo_id = fields.Many2one('zoo.habitaculos', string='Habitáculo', required=True)
    num_animales = fields.Integer(string='Número de animales', required=True, default=1)

    def action_crear_grupo(self):
        if self.num_animales < 1:
            raise ValidationError('Debe haber al menos un animal en el grupo.')

        animales = []
        for i in range(1, self.num_animales + 1):
            nombre_animal = f"{self.name} {i}"
            animal = self.env['zoo.animales'].create({
                'name': nombre_animal,
                'especie_id': self.especie_id.id,
                'habitaculo_id': self.habitaculo_id.id,
                'fecha_llegada': date.today(),
                'fecha_nacimiento': False,  # Fecha de nacimiento vacía
                'state': 'normal',
            })
            animales.append(animal.id)

        return {
            'type': 'ir.actions.act_window',
            'res_model': 'zoo.animales',
            'view_mode': 'tree,form',
            'domain': [('id', 'in', animales)],
            'target': 'current',
        }

    def action_cancel(self):
        """ Cierra el wizard sin hacer cambios """
        return {'type': 'ir.actions.act_window_close'}
```

Especies

```python
def action_abrir_wizard_alta_animales(self):
        """ Abre el wizard para alta animales a las especies seleccionadas """
        return {
            'type': 'ir.actions.act_window',
            'res_model': 'zoo.alta.animales.wizard',
            'view_mode': 'form',
            'target': 'new',
        }
```

Vista

AltaAnimales

```xml
<odoo>
    <record id="action_zoo_alta_animales_wizard" model="ir.actions.act_window">
        <field name="name">Alta animales de Dieta</field>
        <field name="res_model">zoo.alta.animales.wizard</field>
        <field name="view_mode">form</field>
        <field name="target">new</field>
    </record>

    <record id="view_zoo_alta_animales_wizard" model="ir.ui.view">
        <field name="name">zoo.alta.animales.wizard.form</field>
        <field name="model">zoo.alta.animales.wizard</field>
        <field name="arch" type="xml">
            <form string="Alta animales">
                <group>
                    <field name="name" />
                    <field name="especie_id" widget="many2one_tags" />
                    <field name="habitaculo_id" widget="many2one_tags" />
                    <field name="num_animales" />
                </group>
                <footer>
                    <button name="action_crear_grupo" string="Crear Grupo" type="object"
                        class="btn-danger" />
                    <button name="action_cancel" string="Cancelar" class="btn-secondary"
                        special="cancel" />
                </footer>
            </form>
        </field>
    </record>
</odoo>
```

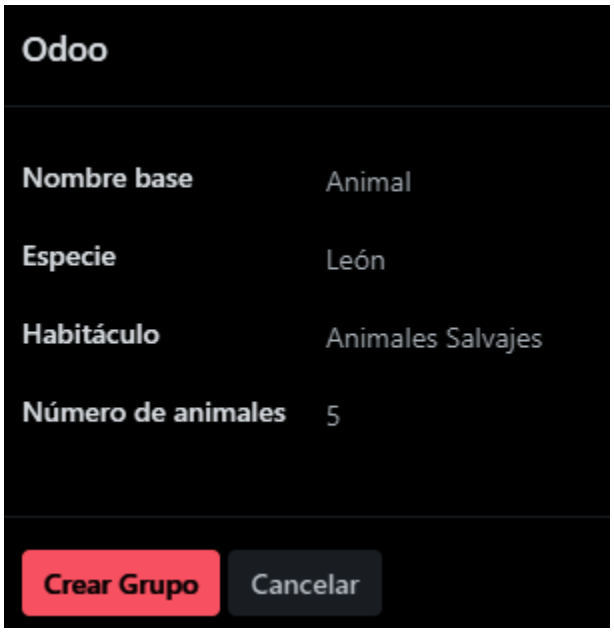
Especies

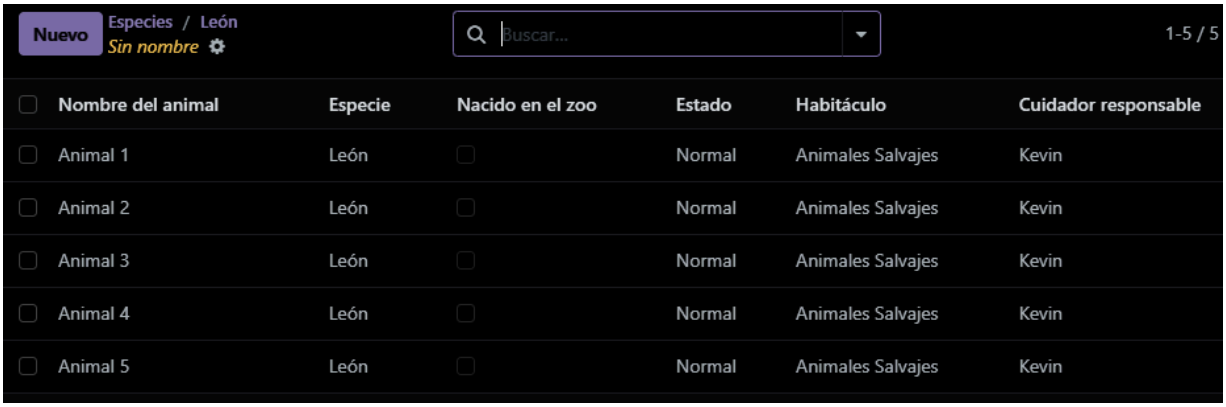
```
<div style="width: 300px; margin: 10px;">
    <button name="action_abrir_wizard_alta_animales"
        string="Alta Animal"
        type="object"
        class="oe_highlight" />
</div>
```

CSV

```
access_zoo_alta_animales_wizard,zoo.alta.animales.wizard,model_zoo_alta_animales_wizard,base.group_user,1,1,1,1
```

Resultado





12.

1. Añadimos registro comida (la comida real que ha comido un animal) como lineas dentro de cada animal. Tenemos los datos fecha, tipo y comida y cantidad.

Modelo
Comida

```
class RegistroComida(models.Model):
    _name = 'zoo.registro.comida'
    _description = 'Registro de comida de los animales'

    animal_id = fields.Many2one('zoo.animales', string='Animal', required=True,
ondelete='cascade')
    fecha = fields.Date(string='Fecha', required=True, default=fields.Date.context_today)
    tipo_comida_id = fields.Many2one('zoo.tipo.comida', string='Tipo de comida', required=True)
    cantidad_kg = fields.Float(string='Cantidad (kg)', required=True, help='Cantidad real
consumida en kilogramos', default=0.0)

    @api.onchange('animal_id')
    def _onchange_animal_id(self):
        """ Muestra solo los tipos de comida de la dieta recomendada y da una advertencia si no
hay dieta definida """
        if self.animal_id:
            dieta_recomendada = self.env['zoo.dieta.recomendada'].search([
                ('especie_id', '=', self.animal_id.especie_id.id)
            ])
            comidas_permitidas = dieta_recomendada.mapped('tipo_comida_id')

            if comidas_permitidas:
                # Asigna el tipo de comida permitido al campo tipo_comida_id
                self.tipo_comida_id = comidas_permitidas[0]  # Selecciona el primer tipo de
comida recomendado
                # Asigna la cantidad recomendada al campo cantidad_kg
                if dieta_recomendada:
                    self.cantidad_kg = dieta_recomendada[0].cantidad_kg  # Toma la cantidad
recomendada de la dieta
                return {'domain': {'tipo_comida_id': [('id', 'in', comidas_permitidas.ids)]}}
            else:
                return {
                    'warning': {
                        'title': "Dieta no definida",
                        'message': "No hay dieta registrada para esta especie. Debes crear una
nueva dieta antes de continuar."
                    }
                }

    @api.onchange('tipo_comida_id')
    def _onchange_tipo_comida_id(self):
```

```python
        """ Da una advertencia si no hay cantidad recomendada para el tipo de comida
seleccionado """
        if self.animal_id and self.tipo_comida_id:
            dieta = self.env['zoo.dieta.recomendada'].search([
                ('especie_id', '=', self.animal_id.especie_id.id),
                ('tipo_comida_id', '=', self.tipo_comida_id.id)
            ], limit=1)

            if not dieta:
                return {
                    'warning': {
                        'title': "Tipo de comida no recomendado",
                        'message': f"{self.tipo_comida_id.name} no está en la dieta recomendada
para esta especie."
                    }
                }
            # Asigna la cantidad recomendada para este tipo de comida
            self.cantidad_kg = dieta.cantidad_kg

    @api.onchange('cantidad_kg')
    def _onchange_cantidad_kg(self):
        """ Da una advertencia si la cantidad de comida no es la recomendada o es 0 """
        if self.animal_id and self.tipo_comida_id:
            # Verifica si la cantidad es 0 o está vacía
            if not self.cantidad_kg or self.cantidad_kg == 0:
                return {
                    'warning': {
                        'title': "Cantidad incorrecta",
                        'message': "La cantidad de comida no puede ser 0 o vacía. Por favor,
introduce una cantidad válida."
                    }
                }

            dieta = self.env['zoo.dieta.recomendada'].search([
                ('especie_id', '=', self.animal_id.especie_id.id),
                ('tipo_comida_id', '=', self.tipo_comida_id.id)
            ], limit=1)

            if dieta and self.cantidad_kg != dieta.cantidad_kg:
                return {
                    'warning': {
                        'title': "Cantidad incorrecta",
                        'message': f"La cantidad recomendada para {self.tipo_comida_id.name} es
{dieta.cantidad_kg} kg, pero has ingresado {self.cantidad_kg} kg."
                    }
                }
```

Animales

```python
registro_comida_id = fields.One2many('zoo.registro.comida', 'animal_id')
```

Vista

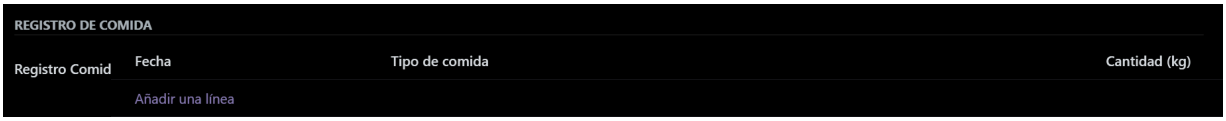
Animales

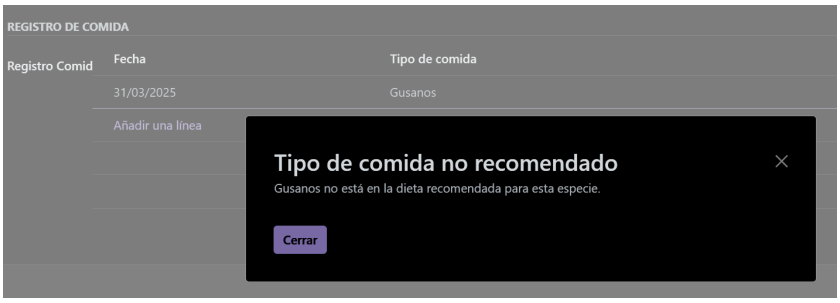
```
<group>
    <separator string="Registro de Comida" colspan="2" />
    <field name="registro_comida_id">
        <tree editable="bottom">
            <field name="fecha" width="100" />
            <field name="tipo_comida_id" width="100" />
            <field name="cantidad_kg" width="100" />
        </tree>
    </field>
</group>
```

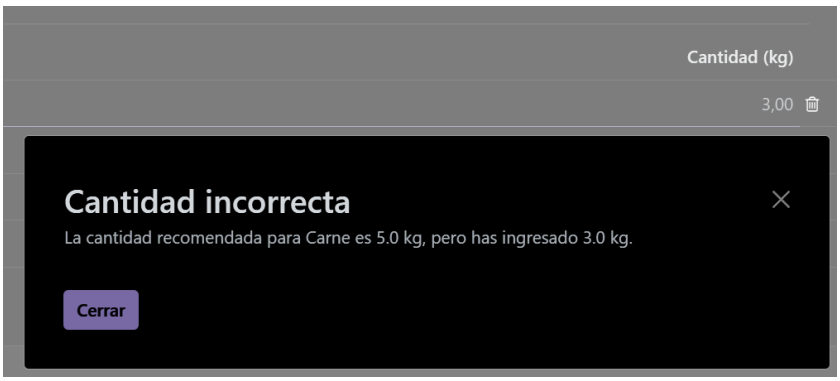
CSV

```
access_zoo_registro_comida,zoo.registro.comida,model_zoo_registro_comida,base.group_user,1,1,1,
1
```

Resultado







2. Wizard dar de comer animales en un habitáculo. Indicamos un habitáculo y una fecha de inicio y de fin. Para cada uno de los días indicados si no se ha dado de comer (no existen registros con esa fecha en su diario de alimentación) añadimos los alimentos de la dieta de su especie.

Modelo
AlimentacionHabitaculoWizard

```
class AlimentacionHabitaculoWizard(models.TransientModel):
    _name = 'zoo.alimentar.habitaculo.wizard'
    _description = 'Registrar alimentación en habitáculo'

    habitaculo_id = fields.Many2one('zoo.habitaculos', string='Habitáculo', required=True)
    fecha_inicio = fields.Date(string='Fecha de inicio', required=True)
    fecha_fin = fields.Date(string='Fecha de fin', required=True)

    def action_alimentar_animales(self):
        RegistroComida = self.env['zoo.registro.comida']
        for wizard in self:
            animales = wizard.habitaculo_id.animal_ids
            fechas = [wizard.fecha_inicio + timedelta(days=i) for i in range((wizard.fecha_fin
- wizard.fecha_inicio).days + 1)]

            # Obtener registros existentes en un solo query
            registros_existentes = RegistroComida.search([
                ('animal_id', 'in', animales.ids),
                ('fecha', 'in', fechas)
            ])
            fechas_registradas = {(r.animal_id.id, r.fecha) for r in registros_existentes}

            # Crear registros solo cuando falten
            nuevos_registros = []
            for fecha in fechas:
                for animal in animales:
                    if (animal.id, fecha) not in fechas_registradas:
                        for alimento in animal.especie_id.dieta_recomendada_ids:
                            nuevos_registros.append({
                                'animal_id': animal.id,
                                'fecha': fecha,
                                'tipo_comida_id': alimento.tipo_comida_id.id,  # ← Corrección
aquí
                                'cantidad_kg': alimento.cantidad_kg,
                            })
            if nuevos_registros:
                RegistroComida.create(nuevos_registros)

        return {'type': 'ir.actions.act_window_close'}
```

Habitaculos

```python
def action_alimentar_animales(self):
    """ Abre el wizard para cambiar el habitáculo de una especie """
    return {
        'name': 'Alimentar Animales del Habitaculo',
        'type': 'ir.actions.act_window',
        'res_model': 'zoo.alimentar.habitaculo.wizard',
        'view_mode': 'form',
        'target': 'new',
    }
```

Vista

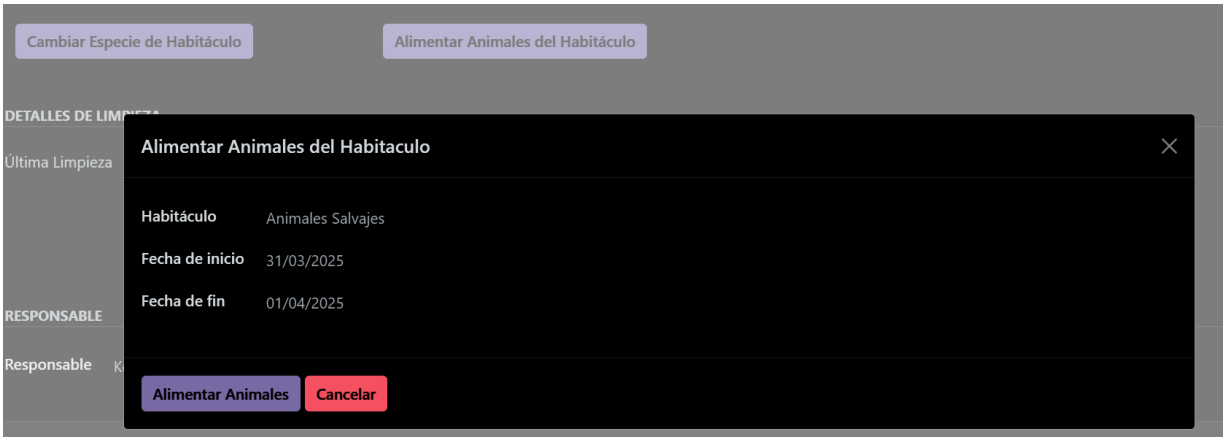
AlimentacionHabitaculoWizard

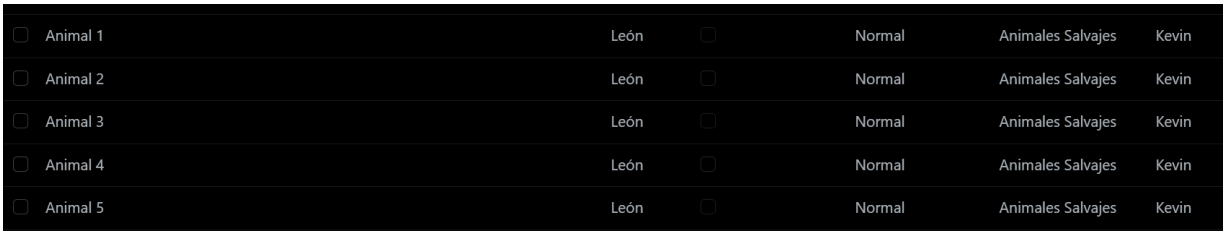
```xml
<odoo>
    <record id="action_zoo_alimentar_habitaculo_wizard" model="ir.actions.act_window">
        <field name="name">Alimentar Animales de los Habitáculos de Dieta</field>
        <field name="res_model">zoo.alimentar.habitaculo.wizard</field>
        <field name="view_mode">form</field>
        <field name="target">new</field>
    </record>

    <record id="view_zoo_alimentar_habitaculo_wizard" model="ir.ui.view">
        <field name="name">zoo.alimentar.habitaculo.wizard.form</field>
        <field name="model">zoo.alimentar.habitaculo.wizard</field>
        <field name="arch" type="xml">
            <form string="Alimentar Animales del Habitaculo">
                <group>
                    <field name="habitaculo_id" />
                    <field name="fecha_inicio" />
                    <field name="fecha_fin" />
                </group>
                <footer>
                    <button name="action_alimentar_animales" string="Alimentar Animales"
type="object" class="btn-primary"
                        />
                    <button name="action_cancel" string="Cancelar" class="btn-danger"
                        special="cancel" />
                </footer>
            </form>
        </field>
    </record>
</odoo>
```

Habitáculos

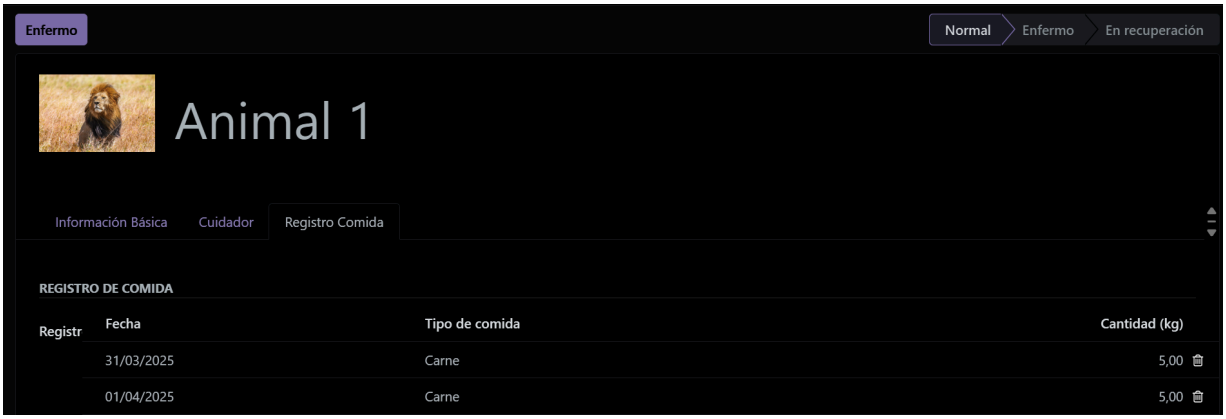
```
<div style="width: 300px; margin: 10px;">
    <button name="action_alimentar_animales"
        string="Alimentar Animales del Habitáculo"
        type="object"
        class="btn-primary" />
</div>
```

Resultado





Animal 1	León		Normal	Animales Salvajes	Kevin
Animal 2	León		Normal	Animales Salvajes	Kevin
Animal 3	León		Normal	Animales Salvajes	Kevin
Animal 4	León		Normal	Animales Salvajes	Kevin
Animal 5	León		Normal	Animales Salvajes	Kevin



13. Añade al menos un par de modelos más para el zoo que usen la estructura cabecera/lineas.

Modelo

HistorialMedico

```
class HistorialMedico(models.Model):
    _name = 'zoo.historial.medico'
    _description = 'Historial médico de los animales'

    name = fields.Char(string='Referencia', required=True, copy=False, default=lambda self:
self.env['ir.sequence'].next_by_code('zoo.historial.medico'))
    animal_id = fields.Many2one('zoo.animales', string='Animal', required=True,
ondelete='cascade')
    fecha_inicio = fields.Date(string='Fecha de inicio', required=True,
default=fields.Date.today)
    fecha_fin = fields.Date(string='Fecha de fin')
    estado = fields.Selection([
        ('abierto', 'Abierto'),
        ('cerrado', 'Cerrado')
    ], string='Estado', default='abierto')
    lineas_historial = fields.One2many('zoo.historial.medico.linea', 'historial_id',
string='Detalles Médicos')

    def action_cerrar(self):
        self.write({'estado': 'cerrado'})
```

HistorialMedicoLinea

```
class HistorialMedicoLinea(models.Model):
    _name = 'zoo.historial.medico.linea'
    _description = 'Entrada en el historial médico'

    historial_id = fields.Many2one('zoo.historial.medico', string='Historial Médico',
required=True, ondelete='cascade')
    fecha = fields.Date(string='Fecha', required=True, default=fields.Date.today)
    descripcion = fields.Text(string='Descripción del tratamiento o diagnóstico')
    veterinario_id = fields.Many2one('zoo.cuidadores', string='Veterinario')
```

Vista

HistorialMedico

```
<odoo>
    <!-- Action para Historial Médico -->
    <record id="action_historial_medico" model="ir.actions.act_window">
        <field name="name">Historial Médico</field>
        <field name="res_model">zoo.historial.medico</field>
        <field name="view_mode">tree,form</field>
    </record>
```

```xml
    <menuitem id="menu_historial_medico"
        name="Historial Médico"
        parent="menu_zoo_config"
        action="action_historial_medico"
        sequence="60" />

    <!-- Tree View -->
    <record id="view_historial_medico_tree" model="ir.ui.view">
        <field name="name">zoo.historial.medico.tree</field>
        <field name="model">zoo.historial.medico</field>
        <field name="arch" type="xml">
            <tree>
                <field name="name" />
                <field name="animal_id" />
                <field name="fecha_inicio" />
                <field name="fecha_fin" />
                <field name="estado" />
            </tree>
        </field>
    </record>

    <!-- Form View -->
    <record id="view_historial_medico_form" model="ir.ui.view">
        <field name="name">zoo.historial.medico.form</field>
        <field name="model">zoo.historial.medico</field>
        <field name="arch" type="xml">
            <form>
                <header>
                    <field name="estado" widget="statusbar" readonly="1" />
                    <button name="action_cerrar" string="Cerrar Historial" type="object"
class="btn-primary" invisible="estado != 'abierto'" />
                </header>
                <sheet>
                    <group>
                        <group colspan="2">
                            <separator string="Información del Historial" colspan="2" />
                            <field name="name" />
                            <field name="animal_id" />
                            <field name="fecha_inicio" />
                            <field name="fecha_fin" />
                        </group>
                    </group>
                    <notebook>
                        <page string="Detalles Médicos">
                            <group>
                                <separator string="Entradas del Historial Médico" colspan="2"
/>
```

```
                                <field name="lineas_historial">
                                    <tree editable="bottom">
                                        <field name="fecha" width="100" />
                                        <field name="descripcion" width="300" />
                                        <field name="veterinario_id" width="150" />
                                    </tree>
                                </field>
                            </group>
                        </page>
                    </notebook>
                </sheet>
            </form>
        </field>
    </record>
</odoo>
```

CSV

```
access_zoo_historial_medico,zoo.historial.medico,model_zoo_historial_medico,base.group_user,1,1
,1,1
access_zoo_historial_medico_linea,zoo.historial.medico.linea,model_zoo_historial_medico_linea,b
ase.group_user,1,1,1,1
```

Resultado

14. Añade además al menos tres Wizards de tu invención. Se valorará la complejidad de los wizards.

Modelo
AsignarCuidadoresWizard

```python
class AsignarCuidadoresWizard(models.TransientModel):
    _name = 'zoo.asignar.cuidadores.wizard'
    _description = 'Wizard para asignar cuidadores a múltiples habitáculos'

    # Campos para selección
    tipo_habitaculo = fields.Selection([
        ('exterior', 'Exterior'),
        ('aviario', 'Aviario'),
        ('granja', 'Granja'),
        ('terrario', 'Terrario'),
        ('acuario', 'Acuario')],
        string='Tipo de habitáculo')

    zona = fields.Char(string='Zona del zoo')
    superficie_min = fields.Float(string='Superficie mínima (m²)')
    superficie_max = fields.Float(string='Superficie máxima (m²)')

    # Campo para asignación
    cuidador_id = fields.Many2one(
        'zoo.cuidadores',
        string='Cuidador a asignar',
        required=True,
        domain="[('estado_laboral_id.orden', '<=', 3)]"  # Solo cuidadores activos
    )

    fecha_efectiva = fields.Date(
        string='Fecha efectiva',
        default=fields.Date.today,
        help="Fecha a partir de la cual se aplicará el cambio"
    )

    # Métodos computados
    total_habitaculos = fields.Integer(
        string='Total de habitáculos afectados',
        compute='_compute_total_habitaculos',
        store=False
    )

    @api.depends('tipo_habitaculo', 'zona', 'superficie_min', 'superficie_max')
    def _compute_total_habitaculos(self):
        domain = self._prepare_domain()
        self.total_habitaculos = self.env['zoo.habitaculos'].search_count(domain)

    def _prepare_domain(self):
```

```python
        domain = []
        if self.tipo_habitaculo:
            domain.append(('tipo', '=', self.tipo_habitaculo))
        if self.zona:
            domain.append(('zona', 'ilike', self.zona))
        if self.superficie_min:
            domain.append(('superficie', '>=', self.superficie_min))
        if self.superficie_max:
            domain.append(('superficie', '<=', self.superficie_max))
        return domain

    def action_asignar_cuidadores(self):
        self.ensure_one()
        domain = self._prepare_domain()
        habitaculos = self.env['zoo.habitaculos'].search(domain)

        if not habitaculos:
            raise UserError("No hay habitáculos que coincidan con los criterios de búsqueda")

        # Asignar nuevo cuidador, eliminando el anterior
        habitaculos.write({'responsable_id': self.cuidador_id.id})

        # Crear notificación
        message = f"""
        Se han actualizado {len(habitaculos)} habitáculos:
        - Nuevo responsable: {self.cuidador_id.name}
        - Fecha efectiva: {self.fecha_efectiva}
        """
        return {
            'type': 'ir.actions.client',
            'tag': 'display_notification',
            'params': {
                'title': 'Asignación completada',
                'message': message,
                'sticky': True,
                'type': 'success',
            }
        }
```

ProgramarLimpiezasWizard

```python
class ProgramarLimpiezasWizard(models.TransientModel):
    _name = 'zoo.programar.limpieza.wizard'
    _description = 'Wizard para programar futuras limpiezas'

    habitaculo_ids = fields.Many2many('zoo.habitaculos', string='Habitáculos', required=True)
    nueva_fecha_limpieza = fields.Date(string='Nueva Fecha de Limpieza',
default=fields.Date.today, readonly=True)
    responsable_id = fields.Many2one('zoo.cuidadores', string='Responsable', required=True)

    def action_confirmar_limpieza(self):
```

```
        """ Actualiza la fecha de limpieza en los habitáculos seleccionados """
        for habitaculo in self.habitaculo_ids:
            habitaculo.write({
                'fecha_limpieza': self.nueva_fecha_limpieza,
                'responsable_id': self.responsable_id.id
            })
        return {'type': 'ir.actions.act_window_close'}
```

AgregarVisitantesEvento

```
class AgregarVisitantesEventoWizard(models.TransientModel):
    _name = 'zoo.agregar.visitantes.evento.wizard'
    _description = 'Asistente para añadir visitantes a un evento'

    evento_id = fields.Many2one(
        'zoo.eventos',
        string='Evento',
        required=True,
        default=lambda self: self._context.get('active_id')
    )
    visitantes_ids = fields.Many2many(
        'zoo.visitantes',
        string='Visitantes a añadir',
        required=True)

    fecha_visita = fields.Date(
        string='Fecha de visita para los nuevos visitantes',
        default=lambda self: fields.Date.today())

    tipo_entrada_evento = fields.Selection(
        related='evento_id.tipo_evento',
        string='Tipo de Evento',
        readonly=True)

    precio_evento = fields.Float(
        related='evento_id.precio',
        string='Precio del Evento',
        readonly=True)

    capacidad_disponible = fields.Integer(
        string='Capacidad disponible',
        compute='_compute_capacidad_disponible')

    @api.depends('evento_id.capacidad_maxima', 'evento_id.visitante_ids', 'visitantes_ids')
    def _compute_capacidad_disponible(self):
        for wizard in self:
            if wizard.evento_id:
                inscritos = len(wizard.evento_id.visitante_ids)
                wizard.capacidad_disponible = wizard.evento_id.capacidad_maxima - inscritos -
len(wizard.visitantes_ids)
            else:
```

```
                wizard.capacidad_disponible = 0

    @api.constrains('visitantes_ids', 'capacidad_disponible')
    def _check_capacidad(self):
        for wizard in self:
            if wizard.capacidad_disponible < 0:
                raise ValidationError('No hay suficiente capacidad para todos los visitantes
seleccionados. Capacidad disponible: %d' % wizard.capacidad_disponible)

    def action_add_visitantes(self):
        self.ensure_one()

        if not self.visitantes_ids:
            raise ValidationError('Debe seleccionar al menos un visitante')

        # Actualizar los visitantes existentes
        self.visitantes_ids.write({
            'fecha_visita': self.fecha_visita,
            'tipo_entrada': 'general',  # O podrías añadir un campo para seleccionar tipo de
entrada
            'total_gastado': self.precio_evento,
        })

        # Relacionar los visitantes con el evento (asumiendo que hay una relación many2many
entre eventos y visitantes)
        # Si no existe, primero deberías añadirla al modelo zoo.eventos:
        # visitante_ids = fields.Many2many('zoo.visitantes', string='Visitantes inscritos')
        self.evento_id.write({
            'visitante_ids': [(4, visitante.id) for visitante in self.visitantes_ids]
        })

        return {'type': 'ir.actions.act_window_close'}
```

Eventos

```
visitante_ids = fields.Many2many('zoo.visitantes', string="Visitantes inscritos")

    inscritos = fields.Integer(
        string="Visitantes Inscritos",
        compute="_compute_inscritos",
        store=True
    )

    @api.depends('visitante_ids')
    def _compute_inscritos(self):
        for evento in self:
            evento.inscritos = len(evento.visitante_ids)
```

Vista

AsignarCuidadoresWizard

```
<?xml version="1.0" encoding="UTF-8"?>
<odoo>
    <!-- Wizard: Asignar Cuidadores -->
    <record id="action_asignar_cuidadores_wizard" model="ir.actions.act_window">
        <field name="name">Asignar Cuidadores a Habitáculos</field>
        <field name="res_model">zoo.asignar.cuidadores.wizard</field>
        <field name="view_mode">form</field>
        <field name="target">new</field>
    </record>

    <!-- Wizard: Asignar Cuidadores -->
    <record id="view_asignar_cuidadores_wizard_form" model="ir.ui.view">
        <field name="name">zoo.asignar.cuidadores.wizard.form</field>
        <field name="model">zoo.asignar.cuidadores.wizard</field>
        <field name="arch" type="xml">
            <form string="Asignar Cuidadores">
                <sheet>
                    <div class="oe_title">
                        <h1>Asignación Masiva de Cuidadores</h1>
                    </div>
                    <group>
                        <group string="Filtros de Habitáculos">
                            <field name="tipo_habitaculo" />
                            <field name="zona" />
                            <field name="superficie_min" />
                            <field name="superficie_max" />
                            <field name="total_habitaculos" widget="progressbar">
                                <progressbar
                                    options="{'current_value': record.total_habitaculos,
'max_value': 100, 'editable': false}" />
                            </field>
                        </group>
                        <group string="Asignación">
                            <field name="cuidador_id" required="1" />
                            <field name="fecha_efectiva" />
                        </group>
                    </group>
                    <footer>
                        <button name="action_asignar_cuidadores" string="Asignar" type="object"
                            class="btn-primary" />
                        <button string="Cancelar" class="btn-secondary" special="cancel" />
                    </footer>
                </sheet>
            </form>
        </field>
    </record>

    <menuitem id="menu_asignar_cuidadores"
        name="Asignar Cuidadores"
```

```
        parent="menu_zoo_config"
        action="action_asignar_cuidadores_wizard"
        sequence="100" />
</odoo>
```

ProgramarLimpiezasWizard

```
<?xml version="1.0" encoding="UTF-8"?>
<odoo>
    <!-- Acción del Wizard -->
    <record id="action_programar_limpieza_wizard" model="ir.actions.act_window">
        <field name="name">Programar Limpieza</field>
        <field name="res_model">zoo.programar.limpieza.wizard</field>
        <field name="view_mode">form</field>
        <field name="target">new</field>
    </record>

    <!-- Vista del Wizard -->
    <record id="view_programar_limpieza_wizard_form" model="ir.ui.view">
        <field name="name">zoo.programar.limpieza.wizard.form</field>
        <field name="model">zoo.programar.limpieza.wizard</field>
        <field name="arch" type="xml">
            <form string="Programar Limpieza">
                <sheet>
                    <div class="oe_title">
                        <h1>Programación Masiva de Limpiezas</h1>
                    </div>
                    <group>
                        <group string="Configuración">
                            <field name="habitaculo_ids" widget="many2many_tags"/>
                            <field name="nueva_fecha_limpieza"/>
                            <field name="responsable_id"/>
                        </group>
                    </group>
                    <footer>
                        <button name="action_confirmar_limpieza" string="Programar"
type="object" class="btn-primary"/>
                        <button string="Cancelar" class="btn-secondary" special="cancel"/>
                    </footer>
                </sheet>
            </form>
        </field>
    </record>

    <!-- Menú de Acceso -->
    <menuitem id="menu_programar_limpieza"
        name="Programar Limpieza"
        parent="menu_zoo_config"
        action="action_programar_limpieza_wizard"
        sequence="110"/>
</odoo>
```

AgregarVisitantesEvento

```xml
<?xml version="1.0" encoding="UTF-8"?>
<odoo>
    <!-- Acción para abrir el asistente -->
    <record id="action_agregar_visitantes_evento_wizard" model="ir.actions.act_window">
        <field name="name">Agregar Visitantes al Evento</field>
        <field name="res_model">zoo.agregar.visitantes.evento.wizard</field>
        <field name="view_mode">form</field>
        <field name="target">new</field>
    </record>

    <!-- Vista del asistente -->
    <record id="view_agregar_visitantes_evento_wizard_form" model="ir.ui.view">
        <field name="name">zoo.agregar.visitantes.evento.wizard.form</field>
        <field name="model">zoo.agregar.visitantes.evento.wizard</field>
        <field name="arch" type="xml">
            <form string="Agregar Visitantes a un Evento">
                <sheet>
                    <div class="oe_title">
                        <h1>Agregar Visitantes al Evento</h1>
                    </div>
                    <group>
                        <group string="Detalles del Evento">
                            <field name="evento_id" />
                            <field name="tipo_entrada_evento" readonly="1" />
                            <field name="precio_evento" readonly="1" />
                            <field name="capacidad_disponible" readonly="1" />
                        </group>
                        <group string="Visitantes">
                            <field name="visitantes_ids" widget="many2many_tags" />
                            <field name="fecha_visita" />
                        </group>
                    </group>
                    <footer>
                        <button name="action_add_visitantes" string="Añadir Visitantes"
type="object" class="btn-primary" />
                        <button string="Cancelar" class="btn-secondary" special="cancel" />
                    </footer>
                </sheet>
            </form>
        </field>
    </record>

    <menuitem id="menu_agregar_visitantes_evento"
        name="Agregar Visitantes"
        parent="menu_zoo_config"
        action="action_agregar_visitantes_evento_wizard"
        sequence="120" />
</odoo>
```

Eventos

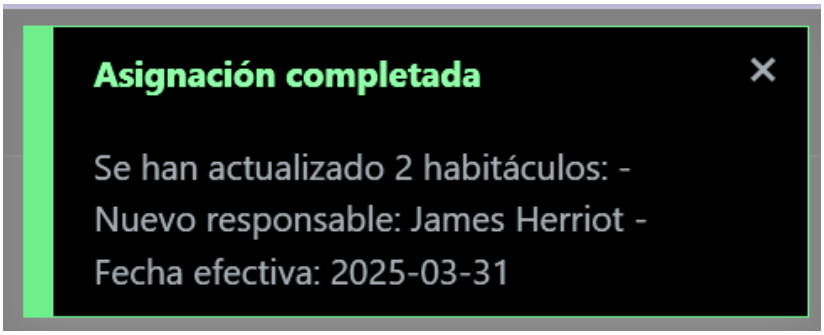
```
<page string="Visitantes">
    <field name="inscritos" />
    <field name="visitante_ids" />
</page>
```

CSV

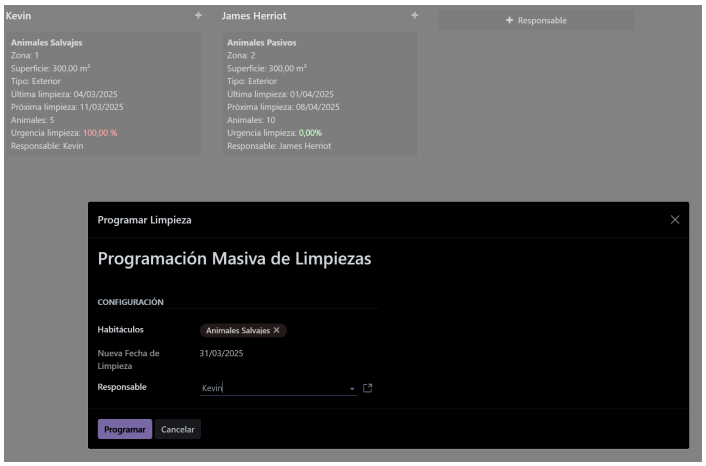
```
access_zoo_asignar_cuidadores_wizard,zoo.asignar.cuidadores.wizard,model_zoo_asignar_cuidadores
_wizard,base.group_user,1,1,1,1
access_zoo_programar_limpiezas_wizard,zoo.programar.limpiezas.wizard,model_zoo_programar_limpie
zas_wizard,base.group_user,1,1,1,1
access_zoo_agregar_visitantes_evento_wizard,zoo.agregar.visitantes.evento.wizard,model_zoo_agre
gar_visitantes_evento_wizard,base.group_user,1,1,1,1
```

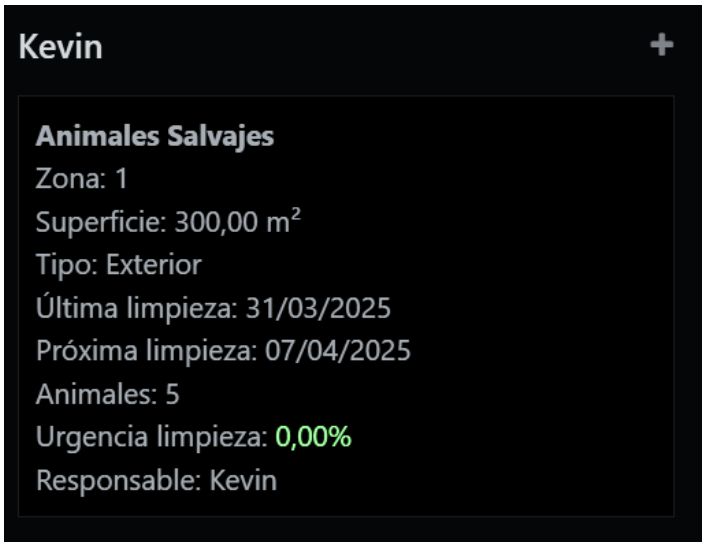
Resultado

AsignarCuidadoresWizard



ProgramarLimpiezasWizard





AgregarVisitantesEvento

Descripción | Requisitos | Visitantes

3

Nombre completo	DNI/Número de identificación	Fecha de visita	Tipo de entrada	Valoración (1-5)	Total gastado	
Fernando Alonso	12345678B	01/04/2025	Entrada General	2	30,00	✖
Pedro Sánchez	98765432C	01/04/2025	Entrada General	0	30,00	✖
Manolo Gómez Valderas	48134118V	01/04/2025	Entrada General	5	30,00	✖

Añadir una línea